

Tendências de Motores de Jogos

Semana 4

GameMode, GameState, PlayerController e GameInstance

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 2 — Construir Sistemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender o Gameplay Framework como conjunto de papéis universais que toda engine precisa resolver
- Diferenciar GameMode (regra da partida) de GameState (estado compartilhado)
- Diferenciar PlayerController (ponte jogador-Pawn) de GameInstance (persistência entre níveis)
- Implementar as quatro classes customizadas no Vertical Slice, com uma variável persistente funcional

Como a semana está organizada



Encontro 1

GameMode (regra da partida) + GameState (estado compartilhado)



Encontro 2

PlayerController (ponte jogador-Pawn) + GameInstance (persistência entre níveis) + primeiro desafio de solução aberta

Tendências de Motores de Jogos

ENCONTRO 1

GameMode e GameState

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Quem decide as regras de uma partida, e quem guarda o que os outros sistemas precisam saber sobre ela?

Pense em um jogo com vida, pontuação ou condição de vitória: onde esses dados deveriam morar?

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

GameMode: regra da partida

- Classe nativa que concentra a lógica autoritativa de uma partida
- Decide como a partida começa, quando termina e as condições de vitória/derrota
- Existe apenas no servidor — mesmo em um projeto single-player, a arquitetura mantém essa separação
- Nenhuma regra de partida deveria estar espalhada dentro do BP_Player



DICA

Um lugar único que toda engine estruturada precisa ter, com ou sem múltiplos jogadores.

GameState: estado compartilhado

- Classe nativa que replica, para todos os clientes, um retrato do estado da partida
- Qualquer sistema — UI, IA, outro jogador — consulta o GameState, nunca o GameMode diretamente
- Critério prático: se o dado precisa ser lido por outros sistemas, é GameState; se é decisão de como a partida se desenrola, é GameMode



DICA

GameMode decide; GameState informa.

Regra e estado no nível de teste



Regra e estado: Unreal × Unity

Unreal

GameMode e GameState são classes nativas, com lugar único e nomeado desde a criação do projeto

Unity

Sem equivalente nativo — o mesmo problema é resolvido por convenção, tipicamente um GameManager singleton (MonoBehaviour com instância estática) ou um ScriptableObject de estado

Demonstração: BP_GameMode e BP_GameState

O professor cria `BP_GameMode` (Game Mode Base) e `BP_GameState` (Game State Base) na pasta `Framework/`, com uma variável de exemplo em cada um.

Resultado esperado: as duas classes vinculadas entre si e atribuídas ao nível de teste via Project Settings > Maps & Modes.

Imagem sugerida

Objetivo: mostrar o campo "Game State Class" no painel Class Defaults do BP_GameMode.

Enquadramento: editor de Blueprint do BP_GameMode, painel Class Defaults visível.

Elementos presentes: campo "Game State Class" preenchido com BP_GameState; painel Variables ao lado com a variável de regra criada.

Destaque visual: contorno ao redor do campo "Game State Class".

Legenda sugerida: "O vínculo entre GameMode e GameState é explícito — nenhum dos dois assume o outro por padrão."

Arquitetura: Framework no projeto

- Blueprints/Framework/BP_GameMode e BP_GameState — primeiras classes da subpasta Framework/
- Project Settings > Maps & Modes aponta para BP_GameMode como Default GameMode
- Base sobre a qual PlayerController e GameInstance serão anexados no Encontro 2

NA INDÚSTRIA

Um ponto único de verdade sobre qual GameMode está de fato ativo no nível — independentemente do que existe no Content Browser.

Boas práticas

✓ BOAS PRÁTICAS

- Nomear a variável de regra pela função, não por "Var1" ou "Bool1"
- Aplicar sempre o critério "isso precisa ser consultado por outros sistemas?" antes de decidir onde uma variável mora
- Revisar Project Settings > Maps & Modes ao final de qualquer alteração no Gameplay Framework

Laboratório do Encontro 1

Criar `BP_GameMode` e `BP_GameState` na pasta `Framework/`, vincular um ao outro e atribuir ao nível de teste do Vertical Slice.

OBJETIVOS

Critério de sucesso: as duas classes customizadas ativas no nível, confirmadas via Print String ou teste em modo Play, com nomenclatura conforme `PROJECT_ARCHITECTURE.md`.

Fechando o Encontro 1

- GameMode decide a regra da partida; GameState expõe o estado compartilhado
- BP_GameMode e BP_GameState customizados, vinculados e atribuídos ao nível de teste
- Próximo encontro: a ponte com o jogador e a persistência entre níveis

Tendências de Motores de Jogos

ENCONTRO 2

PlayerController e GameInstance

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



O que acontece com a maioria dos objetos de um nível quando o jogador troca de mapa?

Pense em progresso, pontuação ou configurações que precisam sobreviver a essa troca.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

PlayerController: jogador × corpo

- Representa a identidade e o input de alto nível do jogador — não a locomoção
- O Pawn/Character (`BP_Player`) é apenas o corpo que ele possui, e pode trocar ou perder sem deixar de existir
- Input não relacionado a locomoção direta (abrir inventário, pausar) pertence ao PlayerController, não ao Pawn



DICA

O mesmo problema que toda engine com posseção de personagem precisa resolver.

GameInstance: o que sobrevive à troca de nível

- Única instância que existe do lançamento ao fechamento do jogo
- Por padrão, Pawn, PlayerController, GameMode e GameState são destruídos a cada troca de nível — o GameInstance não
- Guarda dados que não pertencem a nenhum nível específico: progresso, pontuação, configurações

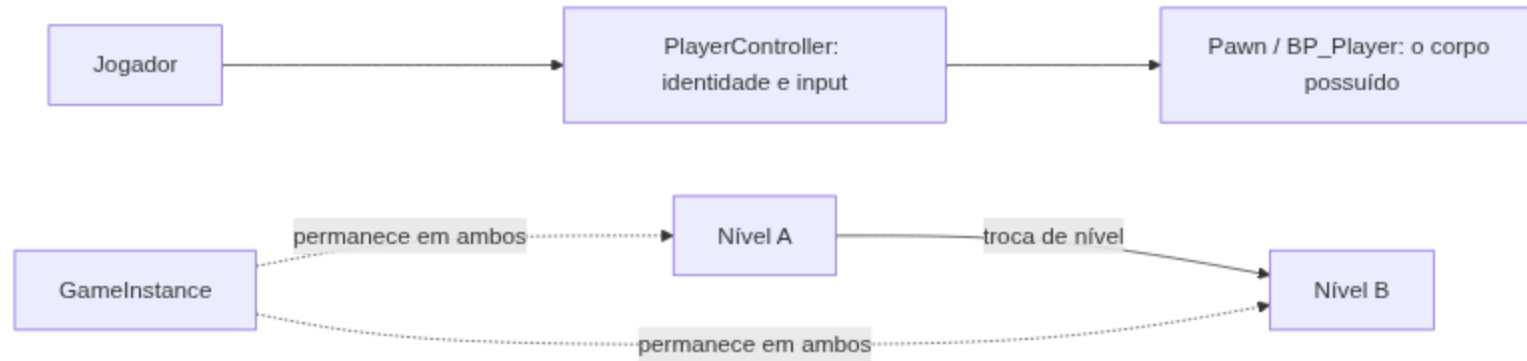


DICA

Pergunte primeiro "isso sobrevive à troca de nível?" — só depois "isso é compartilhado entre sistemas?".

Tendências de Motores de Jogos

Jogador, corpo e persistência



Jogador, corpo e persistência: Unreal × Unity

Unreal

PlayerController separa nativamente identidade de corpo; GameInstance é uma classe dedicada e única

Unity

Corpo e controle normalmente no mesmo GameObject; persistência via DontDestroyOnLoad ou ScriptableObject, por convenção do time

Demonstração: BP_PlayerController

O professor cria `BP_PlayerController`, atribui-o ao `BP_GameMode` e confirma, via Print String, que o BP_Player continua controlável normalmente.

Por que: provar que a troca de PlayerController não altera locomoção, já resolvida na Semana 2.

Demonstração: BP_GameInstance e persistência

O professor cria `BP_GameInstance` com uma variável simples, atribui nas Project Settings, cria um segundo nível mínimo e prova a persistência trocando de nível em modo Play.

Resultado esperado: o valor da variável não reinicia ao trocar de nível.

Imagem sugerida

Objetivo: mostrar a troca de nível em modo Play com o valor persistente exibido em tela.

Enquadramento: duas capturas lado a lado — nível de teste principal e o segundo nível mínimo — cada uma com um Print String do contador do GameInstance.

Elementos presentes: valor do contador visível nas duas capturas, com o segundo valor maior que o primeiro (prova de continuidade, não de reinício).

Destaque visual: seta ligando as duas capturas, indicando a troca de nível.

Legenda sugerida: "O valor não reinicia — o GameInstance sobreviveu à troca de nível."

Arquitetura: Framework completo

- Framework/ agora reúne BP_GameMode , BP_GameState , BP_PlayerController e BP_GameInstance
- As quatro classes substituem, no projeto, as classes padrão da engine
- Base pronta para Interfaces e Event Dispatchers da Semana 5

NA INDÚSTRIA

O PlayerController já criado hoje será reaproveitado, sem refatoração, para input de alto nível como interagir e pausar.

Boas práticas

✓ BOAS PRÁTICAS

- Manter o PlayerController enxuto neste estágio — funcionalidades futuras (inventário, pausa) reutilizarão esta mesma classe
- Testar persistência sempre trocando de nível em Play, nunca reiniciando a sessão
- Aplicar o critério "sobrevive à troca de nível?" antes de "é compartilhado entre sistemas?"

Laboratório do Encontro 2

Criar `BP_PlayerController` e `BP_GameInstance`, atribuir corretamente e implementar uma variável persistente de exemplo, verificada entre dois níveis.

OBJETIVOS

Critério de sucesso: PlayerController ativo sem alterar locomoção; variável do GameInstance demonstravelmente persistente na troca de nível.

Desafio: um dado próprio e persistente

Cada grupo escolhe e implementa um dado que deve persistir entre níveis — pontuação, item coletado ou estado de progresso — com liberdade total sobre qual dado e como é atualizado.

ATENÇÃO

A persistência via GameState precisa ser demonstrável entre o nível de teste e um segundo nível. Sem gabarito único.

Resumo da Semana 4

- GameMode decide a regra; GameState expõe o estado compartilhado
- PlayerController separa jogador de corpo; GameInstance sobrevive à troca de nível
- Gameplay Framework completo: quatro classes customizadas na pasta Framework/
- Primeiro desafio de solução aberta da disciplina concluído

Próximos passos

DICA

A Semana 5 usa o BP_PlayerController já existente como ponto de entrada de input de alto nível (interagir) e introduz Blueprint Interfaces e Event Dispatchers, para que um objeto interativo do nível comunique sua ativação sem depender diretamente do BP_Player.

Leitura recomendada: Gameplay Framework in Unreal Engine (Epic Games Documentation).